

300 Real SQL Interview Questions Asked at PwC, Deloitte, EY, KPMG, Tredence, Persistent Systems and Accenture & More.

Medium to Advanced SQL Questions (01–300)

1. Find the **second** highest salary **from** the Employee table.

```
SELECT MAX(salary) AS SecondHighestSalary  
FROM employees  
WHERE salary < (  
    SELECT MAX(salary)  
    FROM employees  
);
```

2. Find duplicate records in a **table**.

```
SELECT name, COUNT(*)  
FROM employees  
GROUP BY name  
HAVING COUNT(*) > 1;
```

3. Retrieve employees who earn more than their manager.

```
SELECT e.name AS Employee, e.salary, m.name AS  
Manager, m.salary AS ManagerSalary  
FROM employees e  
JOIN employees m ON e.manager_id = m.id  
WHERE e.salary > m.salary;
```

4. Count employees in each department having more than 5 employees.

```
SELECT department_id, COUNT(*) AS  
num_employees  
FROM employees  
GROUP BY department_id  
HAVING COUNT(*) > 5;
```

5. Find employees who joined in the last 6 months.

```
SELECT *  
FROM employees  
WHERE join_date >= CURRENT_DATE -  
INTERVAL '6 months';
```

6. Get departments with no employees.

```
SELECT d.department_name  
FROM departments d  
LEFT JOIN employees e ON d.department_id =  
e.department_id  
WHERE e.id IS NULL;
```

7. Write a query to find the median salary.

```
SELECT AVG(salary) AS median_salary
```

```

FROM (
  SELECT salary
  FROM employees
  ORDER BY salary
  LIMIT 2 - (SELECT COUNT(*) FROM employees)
% 2
  OFFSET (SELECT (COUNT(*) - 1) / 2 FROM
employees)
) AS median_subquery;

```

8. Running total of salaries by department.

```

SELECT name, department_id, salary,
       SUM(salary) OVER (PARTITION BY
department_id ORDER BY id) AS running_total
FROM employees;

```

9. Find the longest consecutive streak of daily logins for each user.

```

WITH login_dates AS (
  SELECT user_id, login_date,
         login_date - INTERVAL ROW_NUMBER()
OVER (PARTITION BY user_id ORDER BY
login_date) DAY AS grp
  FROM user_logins
)
SELECT user_id, COUNT(*) AS streak_length,
       MIN(login_date) AS start_date, MAX(login_date) AS
end_date
FROM login_dates
GROUP BY user_id, grp

```

ORDER BY streak_length DESC;

10. Recursive query to find the full reporting chain for each employee.

```
WITH RECURSIVE reporting_chain AS (  
  SELECT id, name, manager_id, 1 AS level  
  FROM employees  
  WHERE manager_id IS NULL
```

```
  UNION ALL
```

```
  SELECT e.id, e.name, e.manager_id, rc.level + 1  
  FROM employees e  
  JOIN reporting_chain rc ON e.manager_id = rc.id  
)  
SELECT * FROM reporting_chain ORDER BY level,  
id;
```

11. Write a query to find gaps in a sequence of numbers (missing IDs).

```
SELECT (id + 1) AS missing_id  
FROM employees e1  
WHERE NOT EXISTS (  
  SELECT 1 FROM employees e2 WHERE e2.id =  
  e1.id + 1  
)  
ORDER BY missing_id;
```

12. Calculate cumulative distribution (CDF) of salaries.

```
SELECT name, salary,
```

```
    CUME_DIST() OVER (ORDER BY salary) AS  
salary_cdf  
FROM employees;
```

13. Compare two tables and find rows with differences in any column (all columns).

```
SELECT *  
FROM table1 t1  
FULL OUTER JOIN table2 t2 ON t1.id = t2.id  
WHERE t1.col1 IS DISTINCT FROM t2.col1  
      OR t1.col2 IS DISTINCT FROM t2.col2  
      OR t1.col3 IS DISTINCT FROM t2.col3;
```

14. Write a query to rank employees based on salary with ties handled properly.

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS  
salary_rank  
FROM employees;
```

15. Find customers who have not made any purchase.

```
SELECT c.customer_id, c.name  
FROM customers c  
LEFT JOIN sales s ON c.customer_id = s.customer_id  
WHERE s.sale_id IS NULL;
```

16. Write a query to perform a conditional aggregation (count males and females in each department).

```
SELECT department_id,
```

```
COUNT(CASE WHEN gender = 'M' THEN 1
END) AS male_count,
COUNT(CASE WHEN gender = 'F' THEN 1
END) AS female_count
FROM employees
GROUP BY department_id;
```

17. Write a query to calculate the difference between current row and previous row's salary (lag function).

```
SELECT name, salary,
       salary - LAG(salary) OVER (ORDER BY id) AS
salary_diff
FROM employees;
```

18. Identify overlapping date ranges for bookings.

```
SELECT b1.booking_id, b2.booking_id
FROM bookings b1
JOIN bookings b2 ON b1.booking_id <> b2.booking_id
WHERE b1.start_date <= b2.end_date
AND b1.end_date >= b2.start_date;
```

19. Write a query to find employees with salary greater than average salary in the entire company, ordered by salary descending.

```
SELECT name, salary
FROM employees
```

WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;

20. Aggregate JSON data (if supported) to list all employee names in a department as a JSON array.

```
SELECT department_id, JSON_AGG(name) AS  
employee_names  
FROM employees  
GROUP BY department_id;
```

21. Find employees who have the same salary as their manager.

```
SELECT e.name AS Employee, e.salary, m.name AS  
Manager  
FROM employees e  
JOIN employees m ON e.manager_id = m.id  
WHERE e.salary = m.salary;
```

22. Write a query to get the first and last purchase date for each customer.

```
SELECT customer_id,  
       MIN(purchase_date) AS first_purchase,  
       MAX(purchase_date) AS last_purchase  
FROM sales  
GROUP BY customer_id;
```

23. Find departments with the highest average salary.

```
WITH avg_salaries AS (  
  SELECT department_id, AVG(salary) AS avg_salary  
  FROM employees  
  GROUP BY department_id  
)  
SELECT *  
FROM avg_salaries  
WHERE avg_salary = (SELECT MAX(avg_salary)  
FROM avg_salaries);
```

24. Write a query to find the number of employees in each job title.

```
SELECT job_title, COUNT(*) AS num_employees  
FROM employees  
GROUP BY job_title;
```

25. Find employees who don't have a department assigned.

```
SELECT *  
FROM employees  
WHERE department_id IS NULL;
```

21. Write a query to find the difference in days between two dates in the same table.

```
SELECT id, DATEDIFF(day, start_date, end_date) AS  
days_difference  
FROM projects;
```

Note: **DATEDIFF** syntax varies — **replace** accordingly (e.g., **DATEDIFF**('day', start_date, end_date) in some systems).

22. Calculate the moving average of salaries over the last 3 employees ordered by hire date.

```
SELECT name, hire_date, salary,  
       AVG(salary) OVER (ORDER BY hire_date  
ROWS BETWEEN 2 PRECEDING AND CURRENT  
ROW) AS moving_avg_salary  
FROM employees;
```

23. Find the most recent purchase per customer using window functions.

```
SELECT *  
FROM (  
  SELECT *,  
         ROW_NUMBER() OVER (PARTITION BY  
customer_id ORDER BY purchase_date DESC) AS rn  
  FROM sales  
) sub  
WHERE rn = 1;
```

24. Detect hierarchical depth of each employee in the org chart.

```
WITH RECURSIVE employee_depth AS (  
  SELECT id, name, manager_id, 1 AS depth  
  FROM employees  
  WHERE manager_id IS NULL
```

UNION ALL

```
SELECT e.id, e.name, e.manager_id, ed.depth + 1
FROM employees e
JOIN employee_depth ed ON e.manager_id = ed.id
)
SELECT * FROM employee_depth;
```

25. Write a query to perform a self-join to find pairs of employees in the same department.

```
SELECT e1.name AS Employee1, e2.name AS
Employee2, e1.department_id
FROM employees e1
JOIN employees e2 ON e1.department_id =
e2.department_id AND e1.id < e2.id;
```

26. Write a query to pivot rows into columns dynamically (if dynamic pivot is not supported, simulate it for fixed values).

```
SELECT
    department_id,
    SUM(CASE WHEN job_title = 'Manager' THEN 1
ELSE 0 END) AS Managers,
    SUM(CASE WHEN job_title = 'Developer' THEN 1
ELSE 0 END) AS Developers,
    SUM(CASE WHEN job_title = 'Tester' THEN 1
ELSE 0 END) AS Testers
FROM employees
GROUP BY department_id;
```

27. Find customers who made purchases in every category available.

```
SELECT customer_id
FROM sales s
GROUP BY customer_id
HAVING COUNT(DISTINCT category_id) =
(SELECT COUNT(DISTINCT category_id) FROM
sales);
```

28. Identify employees who haven't received a salary raise in more than a year.

```
SELECT e.name
FROM employees e
JOIN salary_history sh ON e.id = sh.employee_id
GROUP BY e.id, e.name
HAVING MAX(sh.raise_date) < CURRENT_DATE -
INTERVAL '1 year';
```

29. Write a query to rank salespeople by monthly sales, resetting the rank every month.

```
SELECT salesperson_id, sale_month, total_sales,
       RANK() OVER (PARTITION BY sale_month
ORDER BY total_sales DESC) AS monthly_rank
FROM (
  SELECT salesperson_id, DATE_TRUNC('month',
sale_date) AS sale_month, SUM(amount) AS
total_sales
  FROM sales
  GROUP BY salesperson_id, sale_month
) AS monthly_sales;
```

30. Calculate the **percentage change** in sales compared to the previous **month** for each **product**.

```
SELECT product_id, sale_month, total_sales,  
       (total_sales - LAG(total_sales) OVER  
(PARTITION BY product_id ORDER BY  
sale_month)) * 100.0 /  
       LAG(total_sales) OVER (PARTITION BY  
product_id ORDER BY sale_month) AS pct_change  
FROM (  
  SELECT product_id, DATE_TRUNC('month',  
sale_date) AS sale_month, SUM(amount) AS  
total_sales  
  FROM sales  
  GROUP BY product_id, sale_month  
) monthly_sales;
```

31. Find employees who earn more than the average salary across the company but less than the highest salary in their department.

```
SELECT *  
FROM employees e  
WHERE salary > (SELECT AVG(salary) FROM  
employees)  
AND salary < (  
  SELECT MAX(salary)  
  FROM employees  
  WHERE department_id = e.department_id  
)
```

32. Retrieve the last 5 orders for each customer.

```
SELECT *  
FROM (  
    SELECT o.*, ROW_NUMBER() OVER  
(PARTITION BY customer_id ORDER BY order_date  
DESC) AS rn  
    FROM orders o  
) sub  
WHERE rn <= 5;
```

33. Find employees with no salary changes in the last 2 years.

```
SELECT e.*  
FROM employees e  
LEFT JOIN salary_history sh ON e.id =  
sh.employee_id AND sh.change_date >=  
CURRENT_DATE - INTERVAL '2 years'  
WHERE sh.employee_id IS NULL;
```

34. Find the department with the lowest average salary.

```
SELECT department_id, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department_id  
ORDER BY avg_salary  
LIMIT 1;
```

35. List employees whose names start and end with the same letter.

```
SELECT *
```

```
FROM employees
WHERE LEFT(name, 1) = RIGHT(name, 1);
```

Questions

31. Write a query to detect circular references in employee-manager hierarchy (cycles).

```
WITH RECURSIVE mgr_path (id, manager_id, path)
AS (
    SELECT id, manager_id, ARRAY[id]
    FROM employees
    WHERE manager_id IS NOT NULL

    UNION ALL

    SELECT e.id, e.manager_id, path || e.id
    FROM employees e
    JOIN mgr_path mp ON e.manager_id = mp.id
    WHERE NOT e.id = ANY(path)
)
SELECT DISTINCT id
FROM mgr_path
WHERE id = ANY(path);
```

32. Write a query to get the running total of sales per customer, ordered by sale date.

```
SELECT customer_id, sale_date, amount,
       SUM(amount) OVER (PARTITION BY
customer_id ORDER BY sale_date) AS running_total
FROM sales;
```

33. Find the department-wise salary percentile (e.g., 90th percentile) using window functions.

```
SELECT department_id, salary,  
       PERCENTILE_CONT(0.9) WITHIN GROUP  
(ORDER BY salary) OVER (PARTITION BY  
department_id) AS pct_90_salary  
FROM employees;
```

34. Find employees whose salary is a prime number.

```
WITH primes AS (  
    SELECT generate_series(2, (SELECT MAX(salary)  
FROM employees)) AS num  
    EXCEPT  
    SELECT num FROM (  
        SELECT num, UNNEST(ARRAY(  
            SELECT generate_series(2, FLOOR(SQRT(num)))  
AS divisor  
        )) AS divisor  
        WHERE num % divisor = 0  
    ) AS composite  
)  
SELECT *  
FROM employees  
WHERE salary IN (SELECT num FROM primes);
```

Note: This is a conceptual approach—some databases may not support this syntax fully.

35. Find employees who have worked for multiple departments over time.

```
SELECT employee_id  
FROM employee_department_history  
GROUP BY employee_id  
HAVING COUNT(DISTINCT department_id) > 1;
```

36. Use window function to find the difference between current row's sales and previous row's sales partitioned by product.

```
SELECT product_id, sale_date, amount,  
       amount - LAG(amount) OVER (PARTITION BY  
product_id ORDER BY sale_date) AS sales_diff  
FROM sales;
```

37. Write a query to find all employees who are at the lowest level in the hierarchy (no subordinates).

```
SELECT *  
FROM employees e  
WHERE NOT EXISTS (  
    SELECT 1 FROM employees sub WHERE  
sub.manager_id = e.id  
);
```

38. Find average order value per month and product category.

```
SELECT DATE_TRUNC('month', order_date) AS  
order_month, category_id, AVG(order_value) AS  
avg_order_value  
FROM orders
```

GROUP BY order_month, category_id;

39. Write a query to create a running count of how many employees joined in each year.

```
SELECT join_year, COUNT(*) AS yearly_hires,  
       SUM(COUNT(*)) OVER (ORDER BY join_year)  
AS running_total_hires  
FROM (  
    SELECT EXTRACT(YEAR FROM hire_date) AS  
join_year  
    FROM employees  
) sub  
GROUP BY join_year  
ORDER BY join_year;
```

40. Write a query to find the second most recent order date per customer.

```
SELECT customer_id, order_date  
FROM (  
    SELECT customer_id, order_date,  
           ROW_NUMBER() OVER (PARTITION BY  
customer_id ORDER BY order_date DESC) AS rn  
    FROM orders  
) sub  
WHERE rn = 2;
```

41. Find employees who have never made a sale.

```
SELECT e.id, e.name  
FROM employees e
```

LEFT JOIN sales s ON e.id = s.employee_id
WHERE s.sale_id IS NULL;

42. Find the average tenure of employees by department.

```
SELECT department_id, AVG(DATE_PART('year',  
CURRENT_DATE - hire_date)) AS avg_tenure_years  
FROM employees  
GROUP BY department_id;
```

43. Get employees with salary in the top 10% in their department.

```
SELECT *  
FROM (  
    SELECT e.*, NTILE(10) OVER (PARTITION BY  
department_id ORDER BY salary DESC) AS decile  
    FROM employees e  
) sub  
WHERE decile = 1;
```

44. Find customers who purchased more than once in the same day.

```
SELECT customer_id, purchase_date, COUNT(*) AS  
purchase_count  
FROM sales  
GROUP BY customer_id, purchase_date  
HAVING COUNT(*) > 1;
```

45. List all departments and their employee counts, including departments with zero employees.

```
SELECT d.department_id, d.department_name,  
COUNT(e.id) AS employee_count  
FROM departments d  
LEFT JOIN employees e ON d.department_id =  
e.department_id  
GROUP BY d.department_id, d.department_name;
```

41. Write a query to find duplicate rows based on multiple columns.

```
SELECT column1, column2, COUNT(*)  
FROM table_name  
GROUP BY column1, column2  
HAVING COUNT(*) > 1;
```

42. Write a recursive query to calculate factorial of a number (e.g., 5).

```
WITH RECURSIVE factorial(n, fact) AS (  
    SELECT 1, 1  
    UNION ALL  
    SELECT n + 1, fact * (n + 1)  
    FROM factorial  
    WHERE n < 5  
)  
SELECT fact FROM factorial WHERE n = 5;
```

43. Write a query to calculate the cumulative percentage of total sales per product.

```
SELECT product_id, sale_amount,
```

```
SUM(sale_amount) OVER (ORDER BY
sale_amount DESC) * 100.0 / SUM(sale_amount)
OVER () AS cumulative_pct
FROM sales;
```

44. Write a query to get employees who reported directly or indirectly to a given manager (hierarchy traversal).

```
WITH RECURSIVE reporting AS (
  SELECT id, name, manager_id
  FROM employees
  WHERE manager_id = 101 -- replace 101 with
manager's id
```

```
UNION ALL
```

```
SELECT e.id, e.name, e.manager_id
FROM employees e
INNER JOIN reporting r ON e.manager_id = r.id
)
SELECT * FROM reporting;
```

45. Find the average number of orders per customer and standard deviation.

```
SELECT AVG(order_count) AS avg_orders,
STDDEV(order_count) AS stddev_orders
FROM (
  SELECT customer_id, COUNT(*) AS order_count
  FROM orders
  GROUP BY customer_id
```